

Visualization Dashboards with Grafana

Turning Metrics into Insights for AI Systems

A comprehensive approach to monitoring and visualizing your AI infrastructure metrics for better performance insights and faster troubleshooting.



What Is Grafana?

Grafana is an open-source **analytics & visualization platform** that transforms raw metrics into actionable insights through intuitive dashboards.

It connects seamlessly to multiple data sources including **Prometheus, Loki, Elastic, CloudWatch**, and many others, making it the ideal centralized visualization layer.



Create powerful **dashboards** for infrastructure, model performance, and business KPIs with a rich ecosystem of plugins, alerts, and sharing capabilities.

Why Grafana for AI Infrastructure?



Unified Visualization

Visualize **GPU health, latency, drift metrics** in one centralized place instead of jumping between different tools



Faster Anomaly Detection

Spot issues through visual patterns in charts rather than parsing through raw metrics or logs



Cross-Team Visibility

Enable **ML engineers, SREs, and business stakeholders** to view the same data with different perspectives



Integrated Alerting

Seamlessly works with **Alertmanager, PagerDuty, and Slack** for immediate notifications when metrics breach thresholds

Core Building Blocks

1

Data Sources

Connect to Prometheus for metrics, Loki for logs, Tempo for traces, and many other data providers

2

Panels

Individual visualization components: graphs, gauges, heatmaps, tables, and stat panels

3

Dashboards

Collections of panels organized for specific use cases or monitoring domains

4

Alerts

Define thresholds and notification channels when metrics exceed acceptable values



Example Dashboards for AI



GPU Utilization

- Per-node and cluster-wide GPU usage
- Temperature, power, ECC errors
- Memory allocation tracking



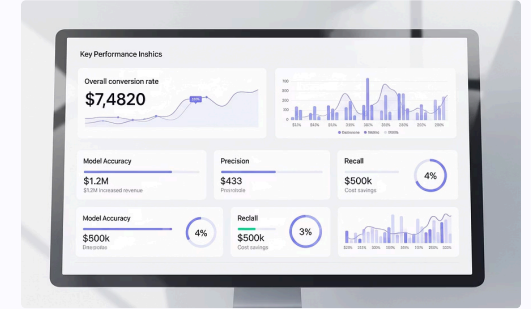
Inference Performance

- QPS, p95/p99 latency, error rates
- Request queue depth monitoring
- Batch size optimization metrics



Data Drift & Quality

- Input distribution histograms
- Feature drift scores over time
- Data quality metrics tracking



Business KPIs

- Conversion rates by model version
- Fraud detection accuracy metrics
- SLA compliance tracking

Prometheus + Grafana Setup

Add Prometheus as Data Source

Configure the connection URL and authentication in Grafana's data source settings

Create PromQL Queries

Build queries to extract metrics for each panel

Configure Auto-Refresh

Set appropriate refresh intervals to keep dashboards current

Example PromQL Query

```
# Request rate by model
rate(nv_inference_count[5m]) by (model)

# P95 latency by instance
histogram_quantile(0.95,
  sum(rate(inference_latency_bucket[5m]))
  by (le, instance))
```


Dashboards in Kubernetes

Deployment Options

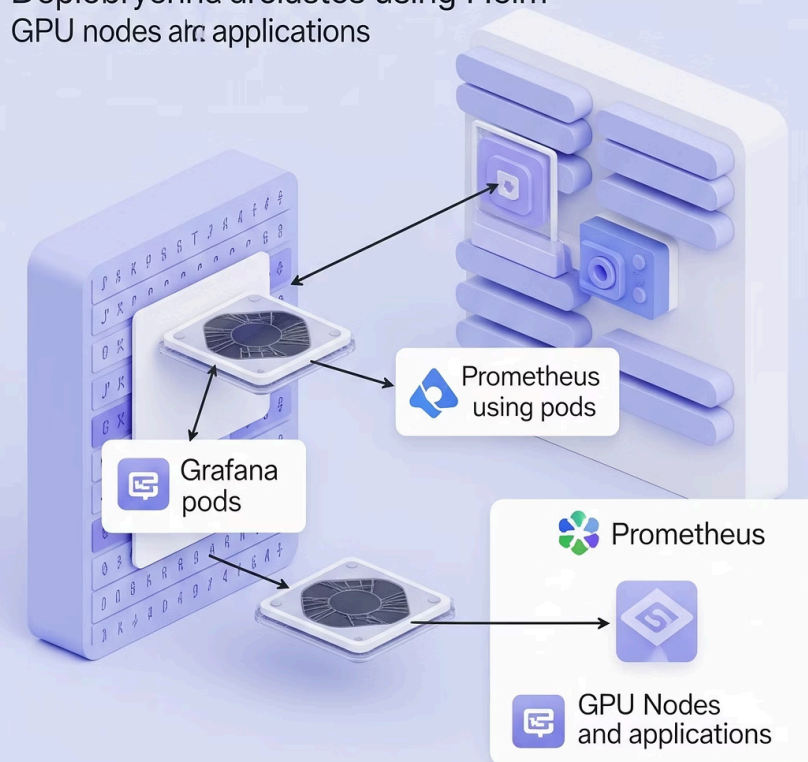
- Run Grafana via **Helm chart** or **Operator** for Kubernetes-native management
- Import NVIDIA GPU + Triton dashboards from community templates
- Configure persistent storage for dashboard definitions

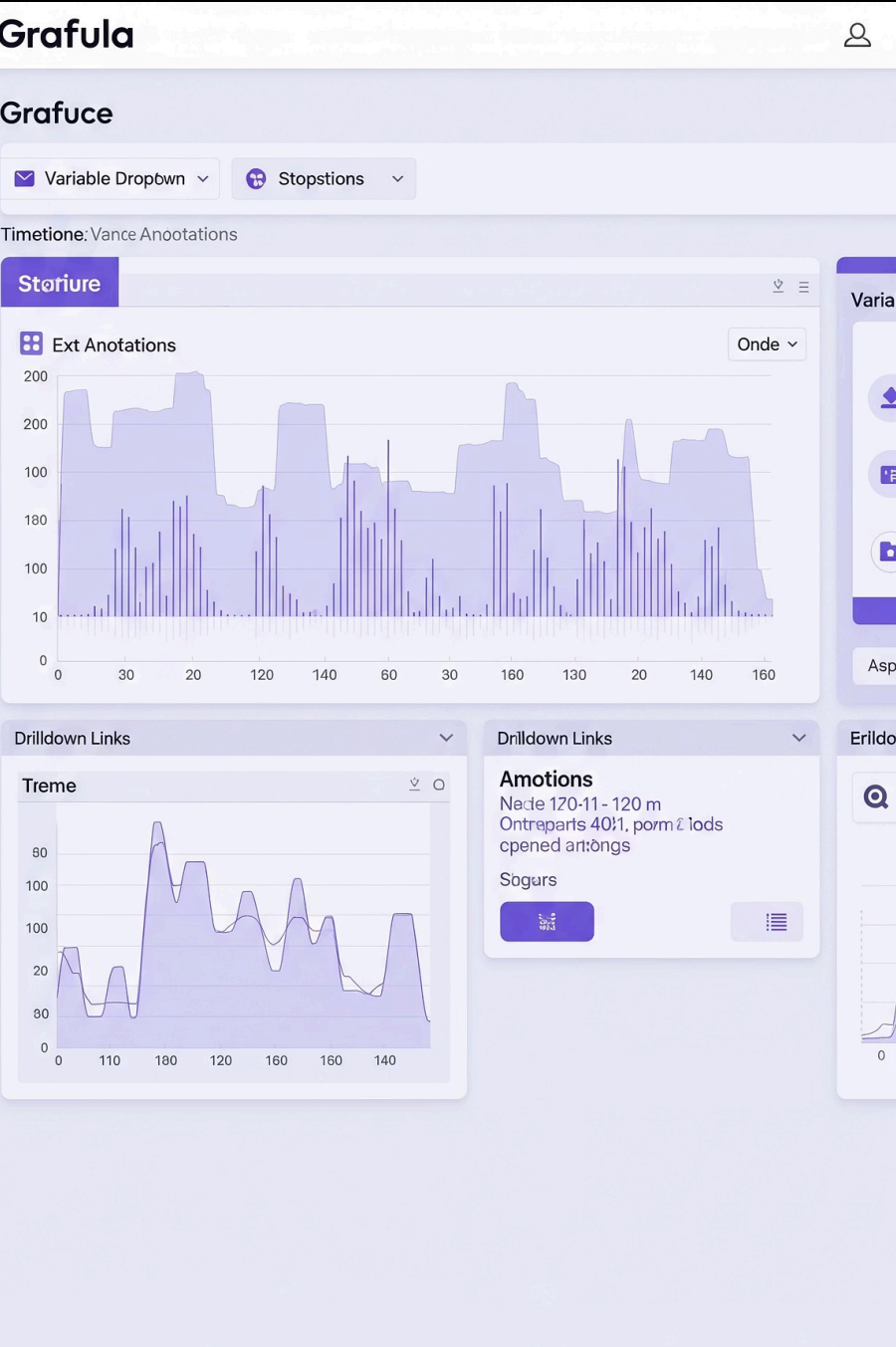
Organization

- Integrate with Kubernetes RBAC for team-based access control
- Use **folders** to group dashboards by domain (infrastructure, ML, business)
- Implement namespace-based segregation for multi-tenant setups

Pro Tip: The kube-prometheus-stack Helm chart automatically deploys Prometheus, Grafana, and default dashboards with best-practice configurations for Kubernetes monitoring.

Deploying clusters using Helm
GPU nodes and applications





Advanced Features

Annotations

Mark important events like deployments, model retraining, and outages directly on time-series graphs
Example: Automatically add annotation via API when new model versions deploy



Variables

Create dropdown selectors to dynamically filter dashboards by cluster, model, or namespace
Example: Single dashboard template that works across dev/staging/prod environments



Drilldowns

Link from high-level cluster view to detailed pod-level metrics with a single click
Example: Jump from model overview to specific instance performance



Alerting 2.0

Configure unified alerts from multiple data sources with sophisticated routing
Example: Alert on correlated metrics from both Prometheus and logs

Best Practices



1 Audience-Focused Design

Define separate dashboards for infrastructure teams, ML engineers, and business stakeholders with appropriate detail levels

2 Prioritize Key Metrics

Focus on **p95/p99 latencies**, GPU saturation, queue depths, and error rates as primary indicators of system health

3 Embrace Templating

Use **variables and template functions** to create reusable dashboards that work across environments and models

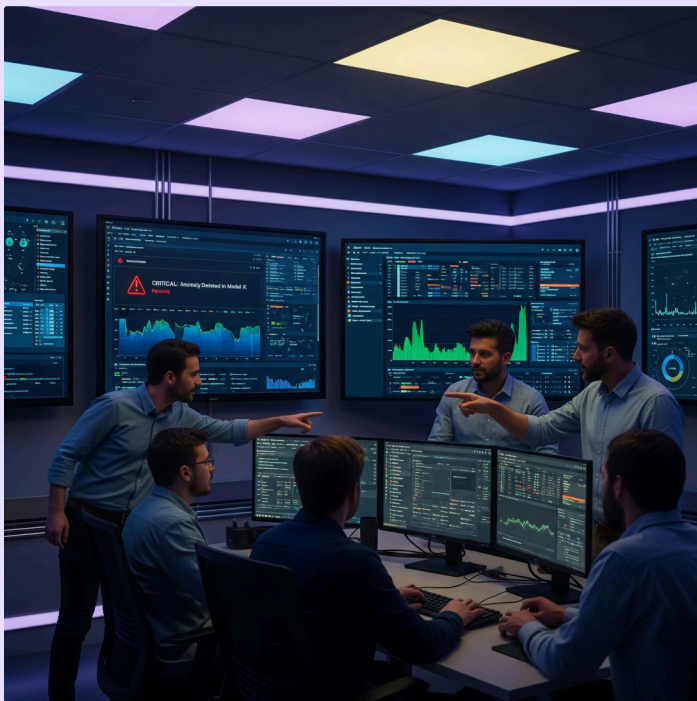
4 Implement GitOps

Store dashboard JSON in version control and automate provisioning via CI/CD pipelines for consistency

⚠ Common Pitfall: Avoid creating too many dashboards with similar metrics. Consolidate related visualizations and use variables to filter rather than duplicating.

Key Takeaways

Grafana: The Eyes of Your AI Infrastructure



Visualization Power

Transforms raw Prometheus metrics into **actionable insights** through intuitive dashboards



Complete Toolkit

Provides **dashboards, alerts, and collaboration** features in a single platform



Essential Component

Must-have for reliable **AI observability stacks** to ensure system health and performance

Next Steps: Start with the NVIDIA GPU dashboard template and customize for your AI infrastructure needs